

Liar's Deck

Sprint 2 Review

Joey Ampfer

Individual Project — ASE 285

Agenda

1. Sprint 2 Goal
2. Demo Recap — Features Delivered (F4–F6)
3. Sprint 2 Metrics & Burndown
4. Lines of Code Breakdown
5. Weekly Progress
6. Retrospective
7. Project Completion & Links
8. Q&A

Sprint 2 Goal

Deliver bluff detection (Call Liar / BS), reconnect-safe hidden state, and MongoDB-backed session persistence with graceful disconnect handling.

Scope

- 3 features planned (F4, F5, F6)
- 9 requirements planned — **7 completed in Sprint 2** (R4.1–R4.3, R5.3, R6.1–R6.3), plus **R5.1 and R5.2** already met in Sprint 1

Demo Recap

Feature 4: Bluff Detection (Call BS / Call Liar)

ID	Requirement	Status
R4.1	Call BS on the previous play (pile reveal)	Done
R4.2	Determine if the declaration was truthful	Done
R4.3	Apply penalties correctly	Done

How it works

- UI: **Call Liar** when it is your turn and `canCallLiar` is true — README requirement uses "Call BS"; same flow
- Server `callLiar` : compares last played cards to `declaredRank` (jokers wild); **successful** vs **unsuccessful** emit drives roulette penalty via `playRoulette`
- `newRound` / win condition when one player remains; state persisted after resolution

Feature 5: Hidden State and Partial Views (R5.3)

ID	Requirement	Status
R5.1	Only receive state data you are allowed to see	Done (Sprint 1)
R5.2	Broadcast public state updates to all players	Done (Sprint 1)
R5.3	No information leaks on reconnect	Done

How it works

- Session token in handshake auth — `tryRestoreSession` reattaches socket to player and room after disconnect
- Broadcasts use `hideGameState` ; full hand only via `getHand` + `showCurrentPlayerHand`
- Socket.IO `connectionStateRecovery` supports short disconnect windows without leaking hands

Feature 6: Session Persistence and Reconnection

ID	Requirement	Status
R6.1	Reconnect to an ongoing game	Done
R6.2	Persist game state to the database	Done
R6.3	Handle player disconnects gracefully	Done

How it works

- `mongoPersistence.ts` : `connectMongo` , `loadRoomSnapshots` on boot, `persistRoomSnapshot` / `deleteRoomSnapshot`
- `schedulePersistRoom` after lobby, play, Call Liar, and game end
- `disconnecting` : notifies room that a user left; empty rooms cleaned up with snapshot removed

Sprint 2 Metrics

Burndown Summary

Metric	Planned	Completed	Burndown
Features	3	3	100%
Requirements	9	9	100%

Tests

- Backend tests: `src/backend/tests/` — `gameLogic.test.js`, `socket.test.js`, `regression.test.js`

Lines of Code Breakdown

File	Lines	Description
src/backend/app.ts	522	Express, Socket.IO, Call Liar, rooms, sessions
src/backend/gameLogic.ts	188	Shuffle, deal, hide/show hands, rounds, roulette
src/backend/mongoPersistence.ts	76	MongoDB connect, load/persist room snapshots
src/backend/index.ts	18	HTTP server entry, graceful Mongo shutdown
src/frontend/src/App.tsx	700	React UI, lobby, game, Call Liar, audio
src/frontend/src/App.css	1243	Layout, cards, playing-table styling
src/frontend/src/socket.ts	9	Socket.IO client configuration
Total	2756	(primary application source)

Weekly Progress

Week	Work Completed
1	Call Liar flow: reveal logic, truth check vs declared rank, UI button and events
2	Penalties (<code>playRoulette</code>), round reset; reconnect privacy (<code>tryRestoreSession</code> , hidden broadcasts)
3	MongoDB persistence — load on startup, persist on state changes, optional <code>MONGODB_URI</code>
4	Graceful disconnect handling, regression/backend tests, UI polish

Retrospective

What Went Wrong

- MongoDB and env configuration took extra setup time locally vs. in-memory-only dev
- Liar / round / elimination edge cases are easy to get wrong — needed careful tracing in `app.ts` and `gameLogic.ts`
- Tests were easier to add after the critical paths stabilized rather than strictly test-first

What Went Well

- End-to-end **F4–F6**: Call Liar, hidden state on reconnect, and disk-backed rooms work together
- Clear split: **game rules** in `gameLogic.ts`, **transport and persistence** in `app.ts` + `mongoPersistence.ts`
- Socket recovery + session tokens give a credible reconnection story for the course demo

Analysis & Improvement Plan

- Document `MONGODB_URI` and local run steps in team onboarding / README for repeatability
- Keep regression tests green when adding UI features (especially around room lifecycle)
- Optional: expand integration coverage for multi-client Call Liar and reconnect races

Project Completion

All Six Features Delivered

Sprint	Features
Sprint 1	F1 Lobby, F2 Dealing, F3 Turn-based play
Sprint 2	F4 Bluff detection, F5 Hidden/reconnect views, F6 Persistence

Course alignment

- **18 requirements** (R1.1–R6.3) mapped in `README.md` — implementation covers the full individual project scope for multiplayer Liar's Deck

Project Links

Resource	Link
GitHub Repository	https://github.com/joseph-ampfer/Bluff_Multiplayer
Project Documentation	<code>docs/</code>
Source Code	<code>src/</code>
Sprint 1 Plan	<code>docs/sprints/sprint1.md</code>
Sprint 2 Plan	<code>docs/sprints/sprint2.md</code>
Sprint 1 Review	<code>docs/presentation/sprint1_review.md</code>
Sprint 2 Review	<code>docs/presentation/sprint2_review.md</code>

Questions?

Thank you!

Liar's Deck

Real-Time Multiplayer Bluffing Card Game